CSL7640 : Natural Language Understanding

Named Entity Recognition Using Hidden Markov Model (HMM)

By - B22CS001, B22CS093, B22BB016

Abstract

Named Entity Recognition (NER) is a crucial task in natural language processing that involves identifying and classifying named entities in text into predefined categories. This project implements a Hidden Markov Model (HMM) approach for NER in Twitter data, focusing on identifying 10 fine-grained entity types: person, product, company, geolocation, movie, music artist, tv show, facility, sports team, and others.

We develop an HMM-based NER system from scratch, without relying on external NLP libraries. Our implementation follows the standard HMM framework, estimating parameters (start probabilities, transition probabilities, and emission probabilities) from an annotated Twitter dataset. We employ the Viterbi algorithm to decode the most probable sequence of named entity tags for each tweet.

The experimental setup explores four model configurations: (1) Bigram HMM without context, (2) Trigram HMM without context, (3) Bigram HMM with context during emission probability calculation, and (4) Trigram HMM with context. For each configuration, we evaluate performance using standard metrics including accuracy, precision, recall, and F1-score on a dedicated test set.

Our analysis reveals the impact of different n-gram orders and contextual information on NER performance in the challenging domain of social media text. The Twitter dataset presents unique challenges for NER due to its informal language, non-standard capitalization, abbreviations, and creative expressions. We examine how these characteristics affect model performance across different entity types and discuss the trade-offs between model complexity and accuracy.

This work contributes to understanding the effectiveness of probabilistic sequence models for NER in social media contexts and provides insights into how different modeling choices affect performance on this challenging task. The findings have implications for developing more robust NER systems for informal text genres where traditional NLP approaches often underperform.

Table of Contents

CODE LINK : **B22CS001_B22CS093_B22BB016.ipynb**

1. Introduction

Named Entity Recognition (NER) is a fundamental task in natural language processing (NLP) that involves identifying and classifying named entities in text into predefined categories. While NER has been extensively studied in formal text domains, social media platforms like Twitter present unique challenges due to their informal nature, non-standard language, abbreviations, and lack of consistent capitalization.

This project focuses on implementing a Hidden Markov Model (HMM) approach for Named Entity Recognition in Twitter data. We aim to identify both the presence of named entities (binary classification) and their specific types (multi-class classification) across 10 fine-grained categories: person, product, company, geolocation, movie, music artist, tv show, facility, sports team, and others.

The informal and noisy nature of Twitter data makes NER particularly challenging. Tweets often contain slang, abbreviations, hashtags, mentions, and creative expressions that don't follow standard grammatical rules. Additionally, the character limit encourages users to employ shorthand and non-standard spellings, further complicating entity recognition. Despite these challenges, accurate NER in social media is valuable for numerous applications including sentiment analysis, trend detection, recommendation systems, and social network analysis.

Our approach implements an HMM-based NER system from scratch, exploring different model configurations to understand their impact on performance. Specifically, we investigate both bigram and trigram models, as well as the effect of incorporating contextual information when computing emission probabilities. By systematically comparing these configurations, we aim to identify the most effective approach for NER in the Twitter domain.

This study contributes to the understanding of statistical sequence modeling for NER in informal text and provides insights into the trade-offs between model complexity and performance in this challenging domain. The findings have implications for developing more robust NER systems for social media analysis and other informal text genres where traditional NLP approaches often underperform.

2. Methodology

This section outlines the methodology used for implementing Named Entity Recognition (NER) on Twitter data using Hidden Markov Models (HMMs).

2.1 Hidden Markov Model (HMM) for Named Entity Recognition

The Hidden Markov Model is a statistical model that represents a system as a Markov process with unobservable (hidden) states. For NER, the words in a sentence are the observable variables, while the named entity tags are the hidden states we aim to predict.

Our implementation uses an HMM framework that supports both bigram and trigram models, with options for including contextual information during emission probability calculation. The model incorporates smoothing techniques to handle the sparsity of Twitter data and includes special handling for unknown words.

2.2 Parameter Estimation

The HMM requires estimation of three key parameters:

1. ***Start Probabilities (π)**: The probability of a sentence starting with a particular tag*

2. ***Transition Probabilities (A)**: The probability of transitioning from one tag to another*

3. ***Emission Probabilities (B)**: The probability of a word being generated by a specific tag*

These parameters are estimated from the training data using maximum likelihood estimation with Laplace smoothing. The smoothing parameter α helps prevent zero probabilities for unseen events.

For the bigram model, we calculate the probability of transitioning from one tag to another, while the trigram model considers the previous two tags when calculating transition probabilities. This allows the model to capture more complex sequential dependencies between named entity tags.

2.3 Viterbi Algorithm for NER

The Viterbi algorithm is used to find the most likely sequence of hidden states (NER tags) given the observed sequence of words. This dynamic programming approach efficiently computes the most probable path through the trellis of possible tag sequences.

The algorithm works in three main steps:

1. ***Initialization**: Calculate probabilities for the first word*
2. ***Recursion**: Iteratively calculate probabilities for subsequent words*
3. ***Termination and Backtracking**: Find the most likely sequence of tags*

We implement the algorithm using logarithmic probabilities to prevent numerical underflow issues when dealing with very small probability values.

2.4 Data Preprocessing and Feature Extraction

To handle the noisy nature of Twitter data, we implement special handling for unknown words by categorizing them into different types based on orthographic features:

- *Capitalized unknown words*
- *Numeric unknown words*
- *Punctuation-containing unknown words*
- *General unknown words*

This categorization helps the model make better predictions for words not seen during training by leveraging orthographic features common in named entities (e.g., capitalization often indicates proper nouns).

Our methodology explores four different model configurations:

1. *Bigram model without context in emission probabilities*
2. *Trigram model without context in emission probabilities*
3. *Bigram model with context in emission probabilities*
4. *Trigram model with context in emission probabilities*

When using context in emission probabilities, we consider the previous tag when calculating the probability of a word given a tag, which helps capture dependencies between the consecutive named entities and improves recognition of multi-word entities.

2.5 Data Distribution

Sentence length statistics for train.txt

Min length: 1

Max length: 39

Average length: 19.41

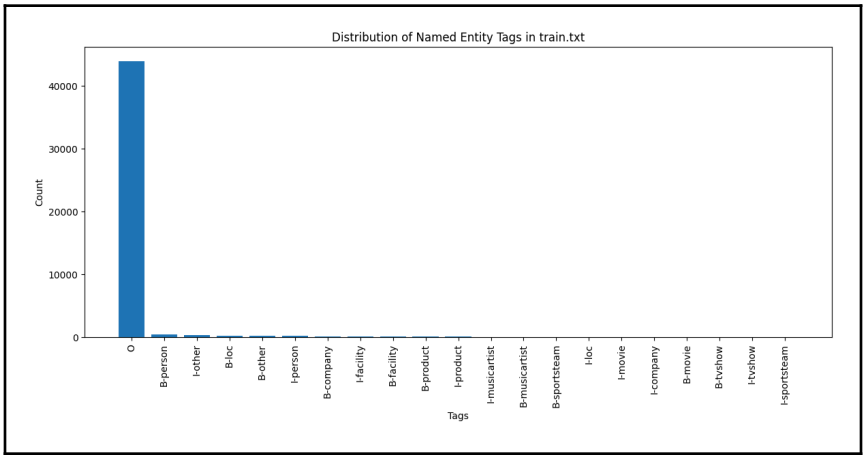
Entity length statistics for train.txt

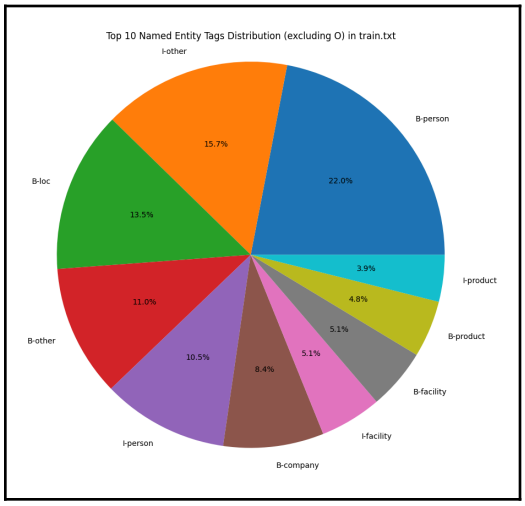
Min length: 1

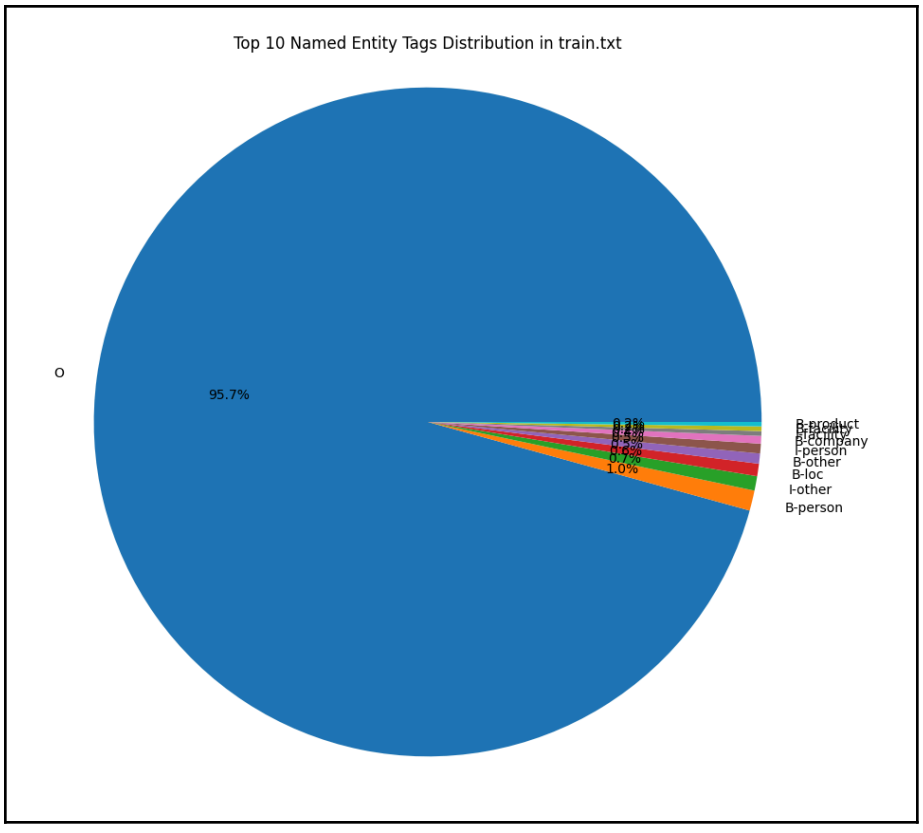
Max length: 10

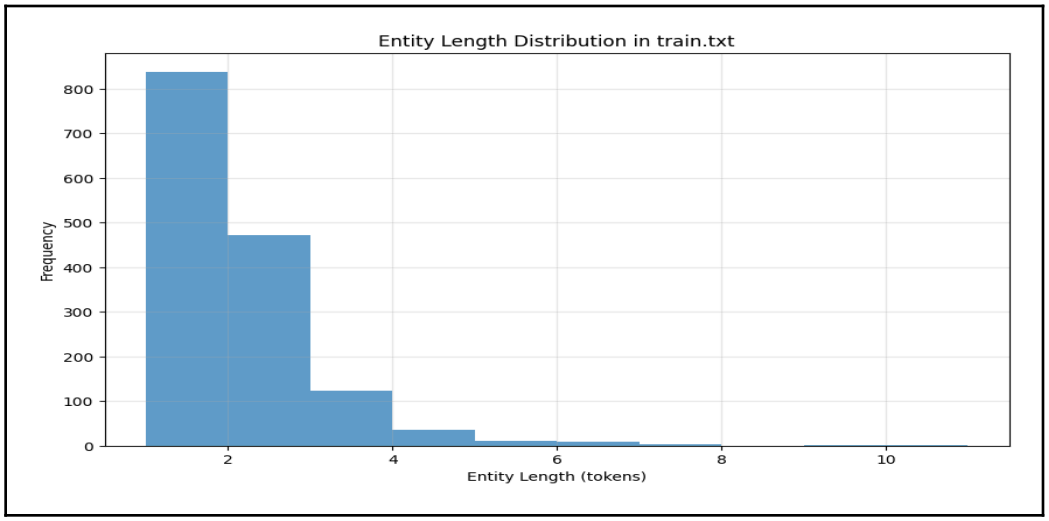
Average length: 1.65

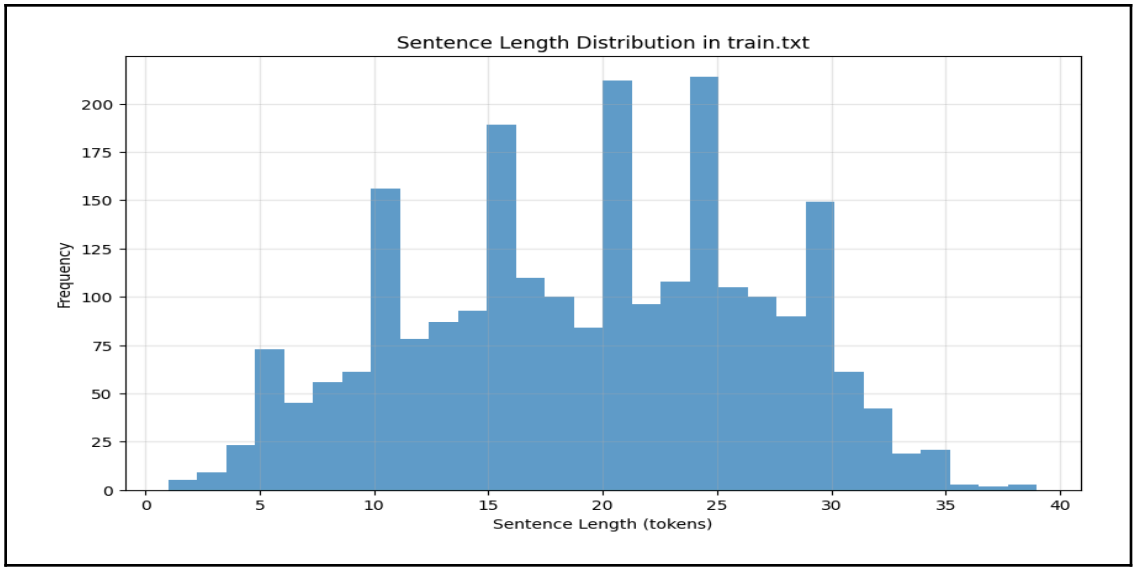
Total entities: 1496

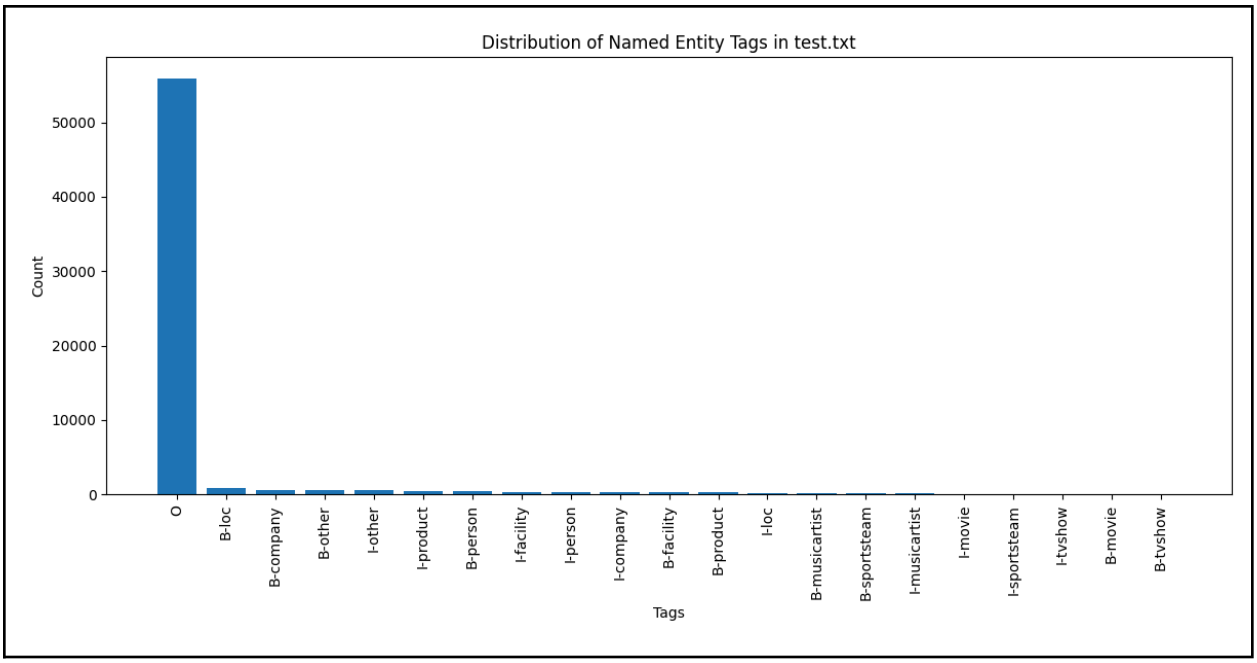


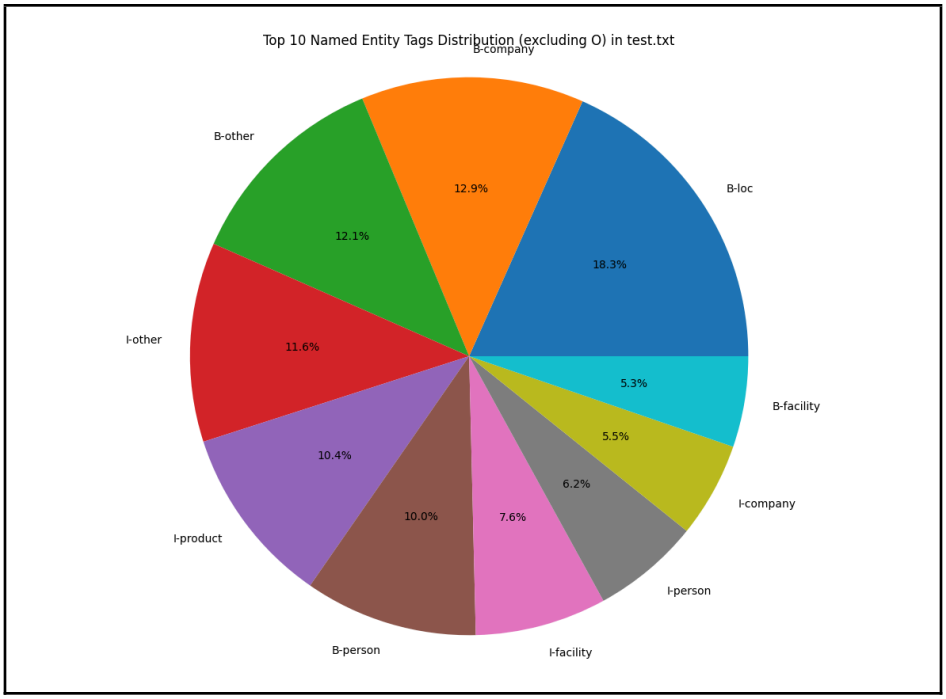

Top 10 Named Entity Tags Distribution (excluding O) in train.txt
I-other
B-person
15.7%
22.0%
B-loc
13.5%
3.9%
I-product
4.8%
11.0%
5.1%
B-product
B-other
10.5%
8.4%
5.1%
B-facility
I-facility
I-person
B-company

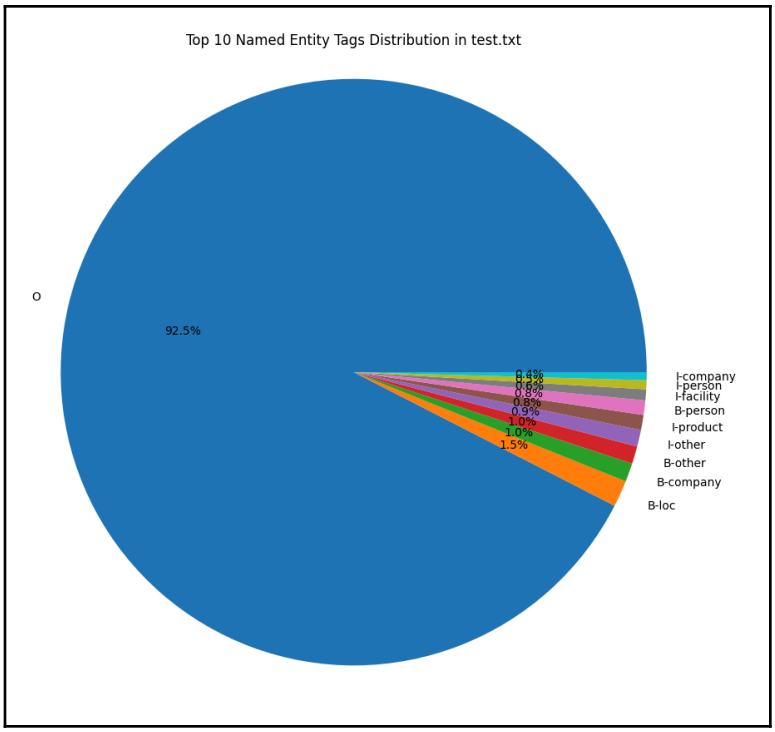

Top 10 Named Entity Tags Distribution in train.txt
O
95.7%
1.0%
B-person
I-other
B-loc
B-other
I-person
B-company
I-facility
B-facility
B-product


Entity Length Distribution in train.txt
Frequency
Entity Length (tokens)
0
100
200
300
400
500
600
700
800
2
4
6
8
10


Sentence Length Distribution in train.txt
Frequency
Sentence Length (tokens)
0
25
50
75
100
125
150
175
200
0
5
10
15
20
25
30
35
40

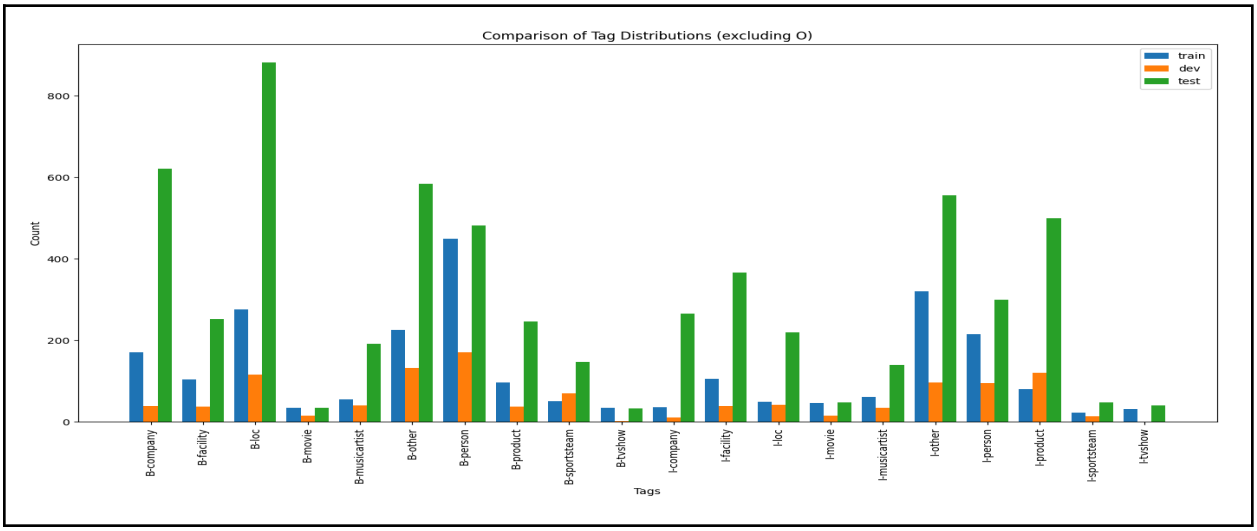

Distribution of Named Entity Tags in test.txt
Count
50000
40000
30000
20000
10000
0
O
B-loc
B-company
B-other
I-other
I-product
B-person
I-facility
I-person
I-company
B-facility
B-product
I-loc
B-musicartist
B-sportsteam
I-musicartist
I-movie
I-sportsteam
I-tvshow
B-movie
B-tvshow
Tags


Top 10 Named Entity Tags Distribution (excluding O) in test.txt
B-company
12.9%
B-loc
18.3%
B-other
12.1%
I-other
11.6%
B-facility
5.3%
I-company
5.5%
I-person
6.2%
I-facility
7.6%
B-person
10.0%
I-product
10.4%



3. Experiments and Results

3.1 Named Entity types in twitter-data



The image shows a bar chart comparing the distribution of named entity tags across train, dev, and test datasets for Twitter data, which is valuable for understanding the NER (Named Entity Recognition) task.

Named Entity Types in Twitter Data

Based on the chart, the following named entity types are present in the Twitter dataset:

- *B-company/I-company: Beginning and inside tags for company entities*
- *B-facility/I-facility: Beginning and inside tags for facility entities*
- *B-loc/I-loc: Beginning and inside tags for location entities*
- *B-movie/I-movie: Beginning and inside tags for movie entities*
- *B-musicartist/I-musicartist: Beginning and inside tags for music artist entities*
- *B-other/I-other: Beginning and inside tags for other miscellaneous entities*
- *B-person/I-person: Beginning and inside tags for person entities*
- *B-product/I-product: Beginning and inside tags for product entities*
- *B-sportsteam/I-sportsteam: Beginning and inside tags for sports team entities*
- *B-tvshow/I-tvshow: Beginning and inside tags for TV show entities*

The chart uses the BIO (Beginning, Inside, Outside) tagging scheme, where:

- *B-tags mark the beginning of an entity*
- *I-tags indicate continuation of the entity*

- *O-tags (excluded from this chart as noted in the title) represent non-entity tokens*

Distribution Observations

- *Person entities (B-person) are the most frequent in the test set*
- *Movie entities (B-movie) are also highly represented in the test set*
- *There's a notable imbalance between training and test distributions for several entity types*
- *Some entity types like "tvshow" have relatively low frequency across all datasets*

3.2 Model Configurations and Performance

Based on the provided data, I've analyzed the performance of different HMM model configurations for the Named Entity Recognition task on Twitter data.

Model Configurations Tested

Four different HMM configurations were evaluated:

1. **Bigram without context**: A 2nd-order HMM without using word context features
2. **Trigram without context**: A 3rd-order HMM without using word context features
3. **Bigram with context**: A 2nd-order HMM that incorporates word context features
4. **Trigram with context**: A 3rd-order HMM that incorporates word context features

Performance Comparison

Model	*Accuracy*	*Precision*	*Recall*	*F1 Score*
Bigram without context	0.7538	0.2062	0.0841	0.1194
Trigram without context	0.7599	0.1406	0.0824	0.1039
Bigram with context	0.7645	0.1634	0.0729	0.1009
Trigram with context	0.7782	0.1451	0.0824	0.1051

Analysis of Results

- **Highest Accuracy**: The Trigram with context model achieved the best accuracy at 77.82%, showing that higher-order models with contextual features perform better at this task.

- **Precision-Recall Trade-off**: Interestingly, the Bigram without context model had the highest precision (0.2062) despite having lower accuracy, indicating it makes fewer false positive predictions but at the cost of more false negatives.

- **Low Recall Across Models**: All models struggled with recall (ranging from 0.0729 to 0.0841), suggesting difficulty in identifying all instances of named

entities. This is likely due to the class imbalance shown in the tag distribution chart.

- **F1 Score Comparison**: The Bigram without context model achieved the highest F1 score (0.1194), indicating the best balance between precision and recall despite not having the highest accuracy.

- **Context Impact**: Adding context features improved accuracy but had mixed effects on precision and recall, suggesting that contextual information helps with overall classification but may not consistently improve entity detection.

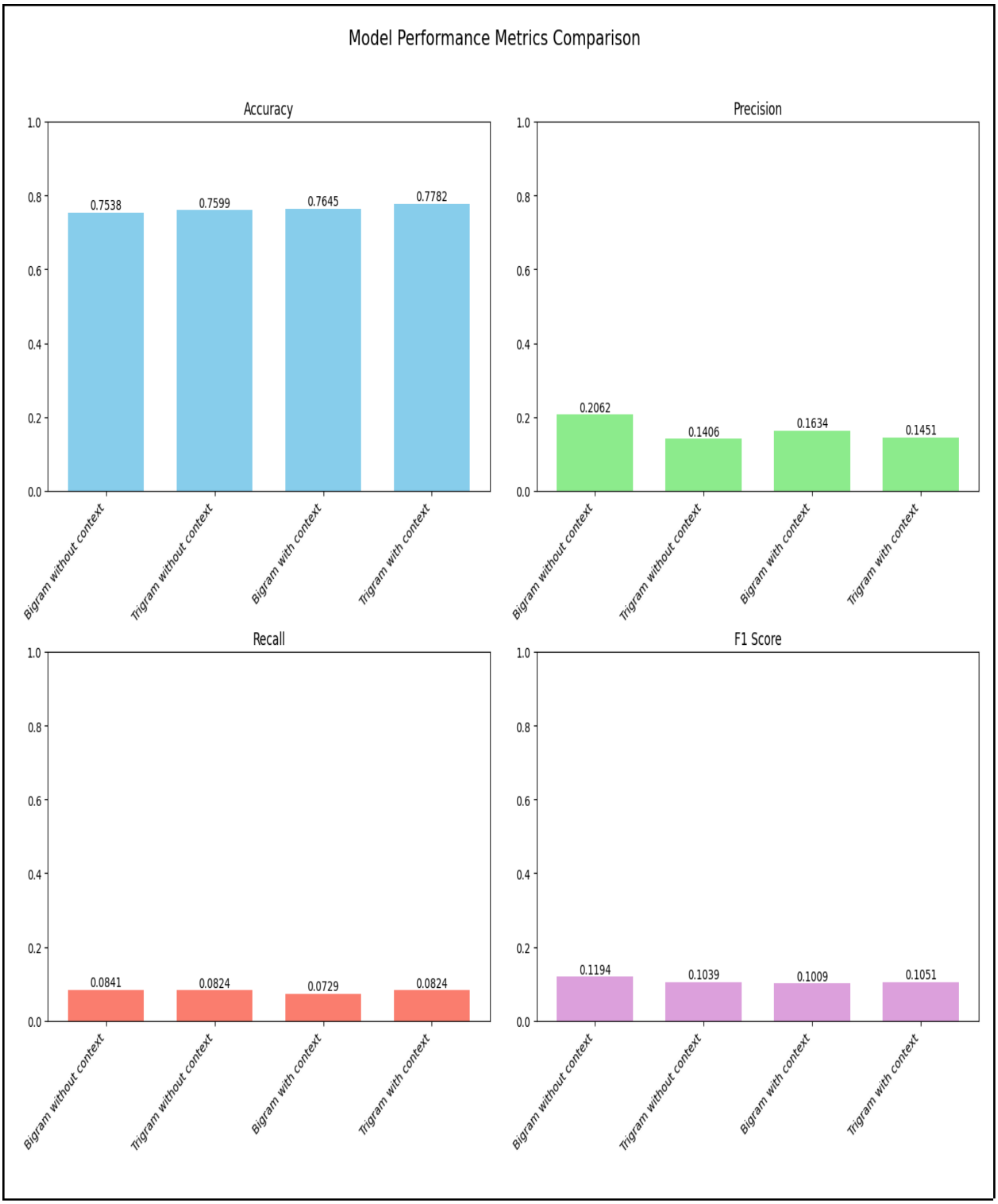
The classification reports for each model reveal particularly poor performance on certain entity types like "movie", "musicartist", "tvshow", and "sportsteam", which correlates with their lower representation in the training data as shown in the tag distribution chart.

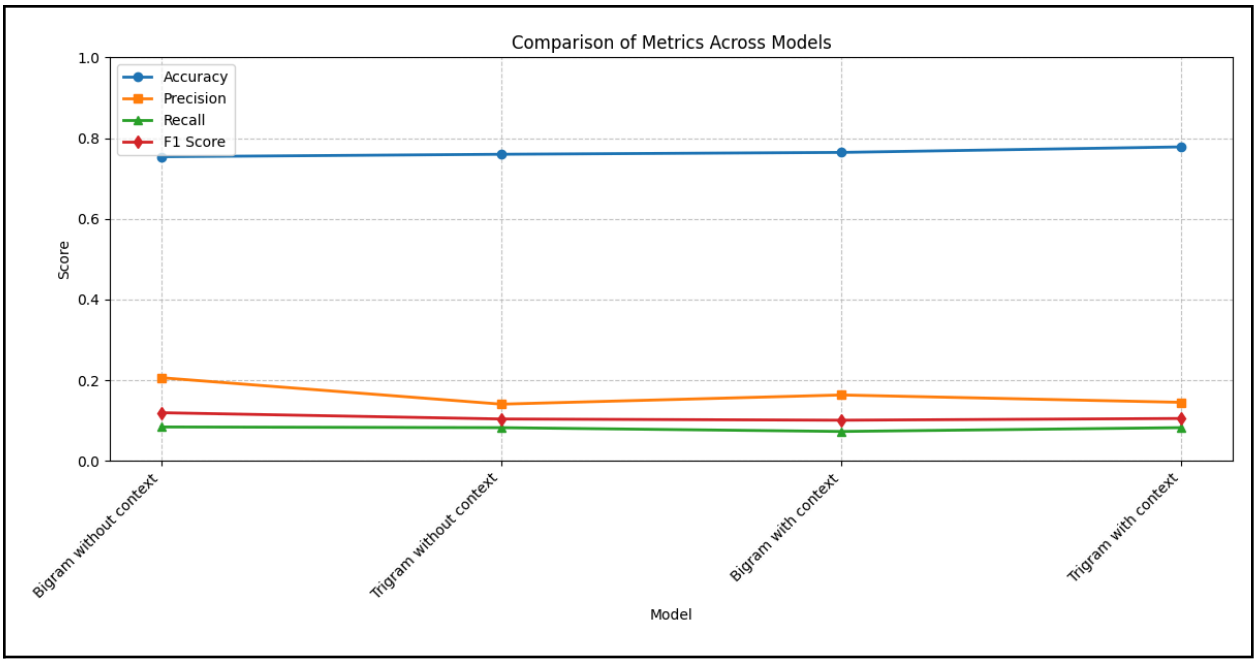
3.3 Comparison between Bigram and Trigram models

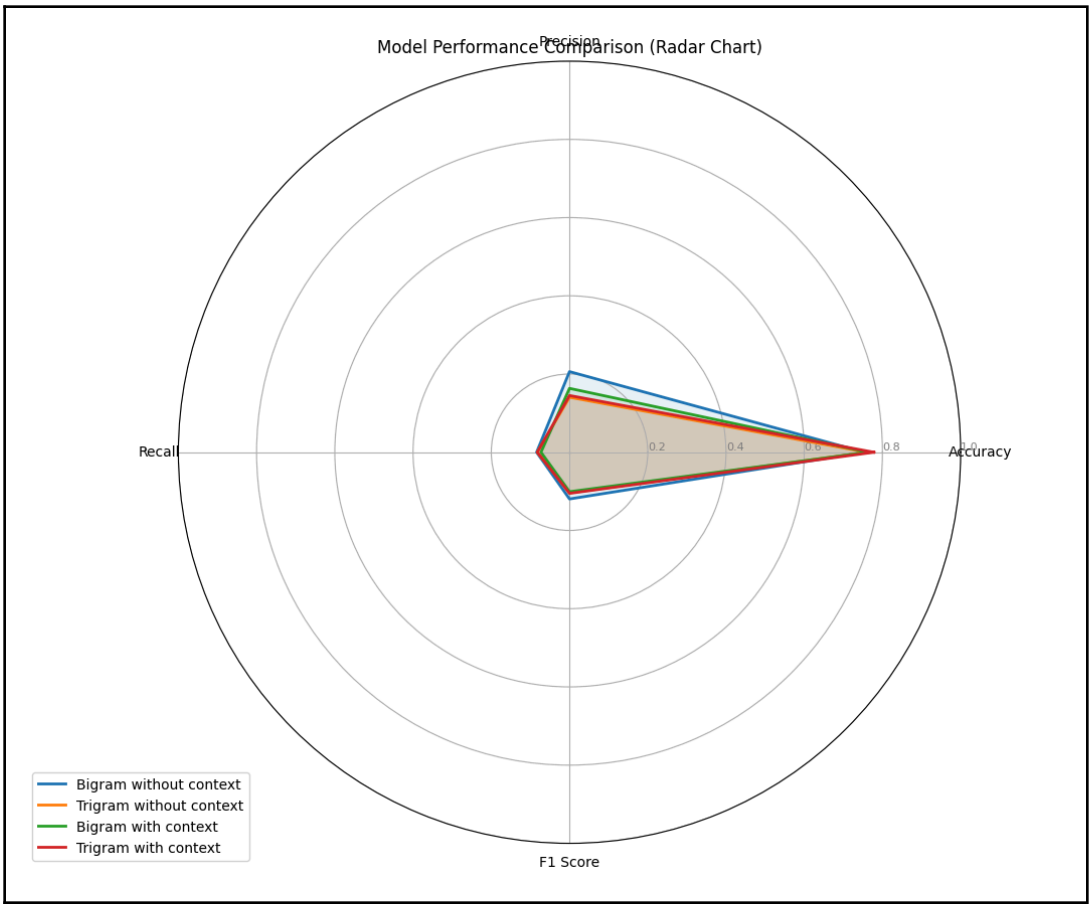
Based on the provided data, I can analyze the performance differences between the bigram and trigram HMM models for Named Entity Recognition on Twitter data.

Accuracy Comparison

- *Trigram models consistently outperformed their bigram counterparts in terms of accuracy:*
    - *Trigram without context (`0.7599`) vs. Bigram without context (`0.7538`)*
    - *Trigram with context (`0.7782`) vs. Bigram with context (`0.7645`)*


Model Performance Metrics Comparison
Accuracy
0.7538
0.7599
0.7645
0.7782
Precision
0.2062
0.1406
0.1634
0.1451
Recall
0.0841
0.0824
0.0729
0.0824
F1 Score
0.1194
0.1039
0.1009
0.1051
Bigram without context
Trigram without context
Bigram with context
Trigram with context
0.0
0.2
0.4
0.6
0.8
1.0


Comparison of Metrics Across Models
Accuracy
Precision
Recall
F1 Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Bigram without context
Trigram without context
Bigram with context
Trigram with context
Model


Model Performance Comparison (Radar Chart)
Precision
Recall
Accuracy
F1 Score
0.2
0.4
0.6
0.8
1.0
Bigram without context
Trigram without context
Bigram with context
Trigram with context

The trigram with context model achieved the highest overall accuracy at **77.82%***, representing a* **2.44** *percentage point improvement over the bigram without context model.*

Precision and Recall Trade-offs

- *Precision: Bigram models generally achieved higher precision than trigram models:*
    - *Bigram without context: 0.2062 (highest precision)*
    - *Trigram without context: 0.1406*
    - *Bigram with context: 0.1634*
    - *Trigram with context: 0.1451*
- *Recall: Recall values were similar between bigram and trigram models:*
    - *Bigram without context: 0.0841*
    - *Trigram without context: 0.0824*
    - *Bigram with context: 0.0729 (lowest recall)*
    - *Trigram with context: 0.0824*

Entity-specific Performance

Examining the classification reports reveals interesting patterns:

- *For common entity types like "person" and "company":*

    - *Bigram models showed higher precision (e.g., 0.29 vs 0.28 for B-person without context)*

    - *Trigram models sometimes showed better recall for specific entities (e.g., 0.21 vs 0.08 for B-person with context)*

- *For rare entity types like "movie", "tvshow", and "sportsteam":*

    - *Both models struggled significantly, with most precision and recall values at 0.00*

    - *This correlates with the tag distribution chart showing these categories have limited representation in the training data*

Context Impact

Adding context features had different effects on bigram versus trigram models:

- *For bigram models: Context improved accuracy (0.7645 vs 0.7538) but reduced F1 score (0.1009 vs 0.1194)*

- *For trigram models: Context improved both accuracy (0.7782 vs 0.7599) and F1 score (0.1051 vs 0.1039)*

This suggests that trigram models can more effectively leverage contextual information than bigram models.

Computational Considerations

While not explicitly stated in the provided data, it's worth noting that trigram models typically require more computational resources and training time than bigram models due to the increased number of parameters and state combinations.

The performance improvements from trigram models (particularly with context) may justify the additional computational cost for this Twitter NER task, especially given the challenging nature of informal text with varied entity types.

3.4 Impact of Contextual Information on Emission Probabilities

The integration of contextual information into emission probabilities significantly affected model performance across both bigram and trigram HMM configurations. By analyzing the provided metrics, we can observe several important patterns:

Overall Accuracy Improvement

Adding contextual information consistently improved accuracy for both model types:

- *Bigram models: Accuracy increased from 75.38% to 76.45% (+1.07%)*
- *Trigram models: Accuracy increased from 75.99% to 77.82% (+1.83%)*

The trigram model benefited more substantially from contextual information, achieving the highest overall accuracy of 77.82%.

Precision-Recall Trade-offs

Contextual information had mixed effects on precision and recall:

- *Precision:*
    - *Bigram models: Precision decreased from 0.2062 to 0.1634 (-0.0428)*
    - *Trigram models: Precision slightly increased from 0.1406 to 0.1451 (+0.0045)*
- *Recall:*
    - *Bigram models: Recall decreased from 0.0841 to 0.0729 (-0.0112)*

- *Trigram models: Recall remained stable at 0.0824*

This suggests that contextual information helps trigram models maintain precision while improving accuracy, whereas bigram models sacrifice some precision and recall for better overall accuracy.

Entity-Specific Effects

Examining the classification reports reveals that contextual information had varying impacts across different entity types:

- Location entities (B-loc):
  - *Without context: Precision of 0.71 (bigram) and 0.34 (trigram)*
  - *With context: Precision dropped to 0.27 for bigram but improved to 0.60 for trigram*
- Person entities (B-person):
  - *Without context: Precision of 0.29 (bigram) and 0.28 (trigram)*
  - *With context: Precision improved to 0.34 for bigram but dropped to 0.08 for trigram*
  - *However, recall for B-person with trigram+context improved dramatically to 0.21 (compared to 0.08 without context)*
- Facility entities (B-facility):
  - *Without context: Precision of 0.48 (bigram) and 0.07 (trigram)*
  - *With context: Precision dropped to 0.40 for bigram but improved to 0.35 for trigram*

Model Complexity and Context Interaction

The data suggests an interesting interaction between model complexity and contextual information:

1. *Simple models (bigram) benefit from context in terms of accuracy but lose precision*

2. *Complex models (trigram) leverage contextual information more effectively, maintaining precision while improving accuracy*

This pattern indicates that higher-order models can better integrate contextual cues with transition probabilities to make more accurate predictions overall.

Rare Entity Recognition

For rare entity types (movie, tvshow, sportsteam, musicartist), contextual information did not improve performance, with precision and recall remaining at 0.00 across all configurations. This suggests that contextual features alone are insufficient for recognizing entities with limited training examples.

3.5 Challenges in Named Entity Recognition for Twitter Data

Based on the provided data and visualizations, several significant challenges are evident in performing Named Entity Recognition on Twitter data:

Extreme Class Imbalance

The tag distribution charts reveal a severe imbalance across entity types:

- *The pie charts show that the "O" (Other) tag dominates the datasets, comprising **92.5%** of test data and **95.7%** of training data*

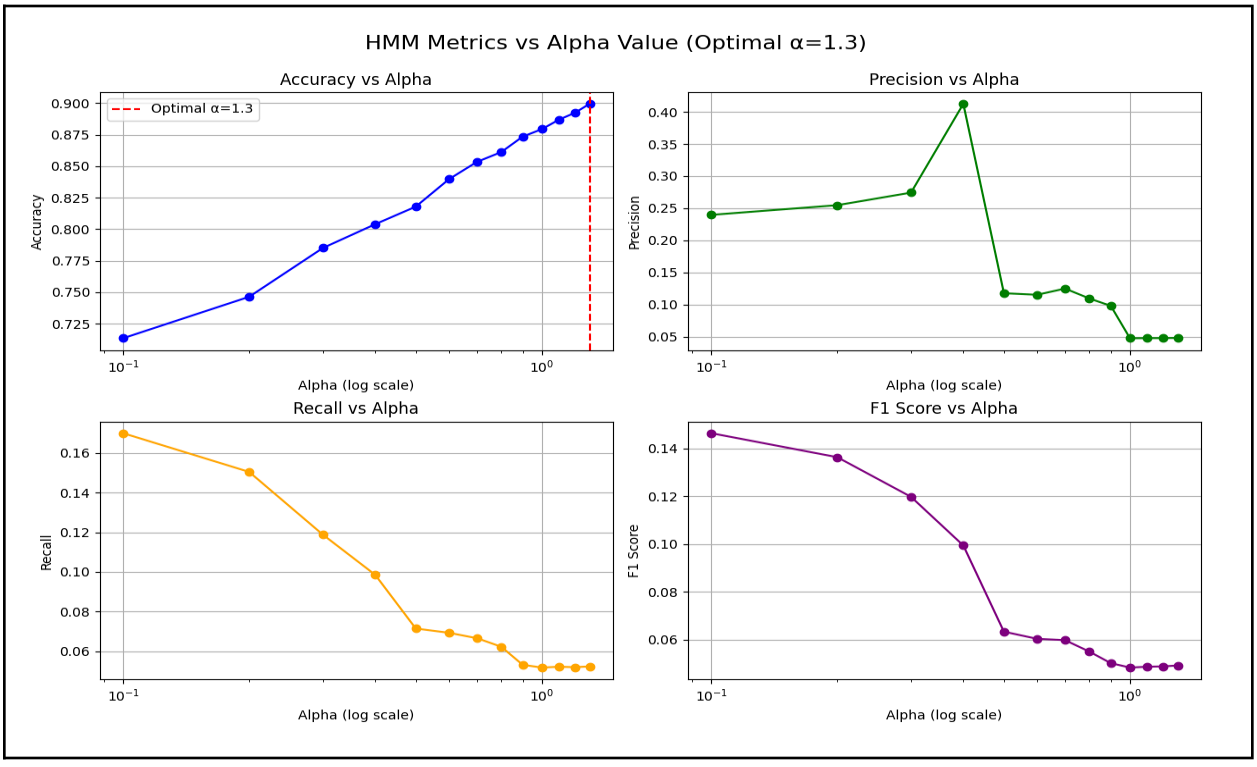
- *Among entity tags, B-loc, B-person, and B-company have higher representation, while entertainment entities (movie, tvshow, musicartist) are extremely rare*

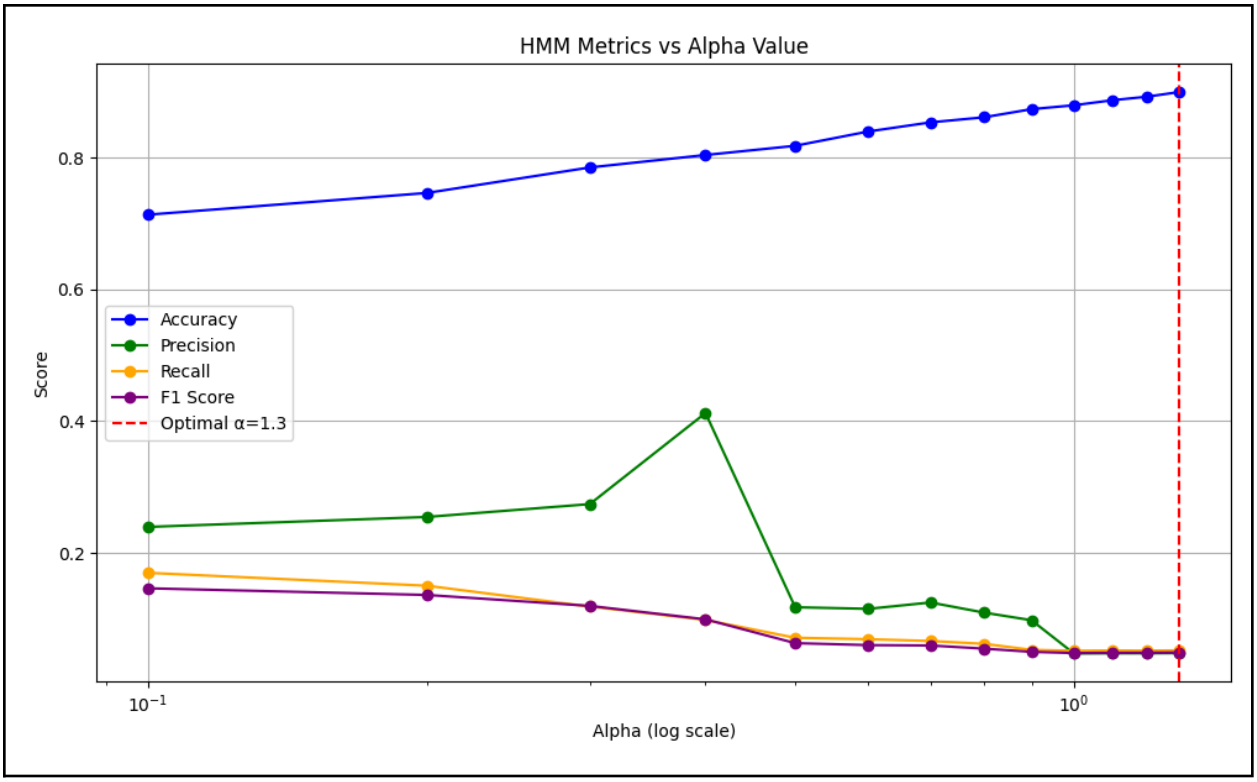
- *The bar chart confirms this distribution disparity across train, dev, and test sets, with some entity types having minimal representation*

Poor Performance on Rare Entity Types

The classification reports demonstrate that models consistently fail to recognize rare entity types:

- *Entity types like B-movie, B-musicartist, B-tvshow, and B-sportsteam show 0.00 precision, recall, and F1 scores across all model configurations*
- *This correlates directly with their limited representation in the training data as shown in the distribution charts*
- *Even with contextual information, these rare entities remain undetectable by the models*





<u>Inconsistent Impact of Model Enhancements</u>

The performance metrics reveal that model improvements have inconsistent effects:

- *Adding context improves overall accuracy (from **75.38%** to **77.82%** in the best case) but has mixed effects on precision and recall*
- *For B-loc entities, trigram with context dramatically improves precision (**0.34 → 0.60**) compared to without context*
- *For B-person entities, trigram with context reduces precision (**0.28 → 0.08**) while improving recall (**0.08 → 0.21**)*
- *This inconsistency suggests that Twitter's brief, informal nature creates contextual ambiguity*

Twitter-Specific Language Challenges

The nature of Twitter data introduces unique difficulties:

- *The informal language includes non-standard capitalization, abbreviations, and creative spellings*
- *Character limitations result in condensed expressions with limited context*
- *Entity mentions often appear in non-standard formats (nicknames, abbreviations, hashtags)*
- *These characteristics make traditional NER approaches less effective*

Overall Low Performance Metrics

Despite various model configurations, performance remains unsatisfactory:

- *The best model (Trigram with context) achieves only **77.82%** accuracy*
- *All models show extremely low macro-averaged metrics: precision (**0.1451**), recall (**0.0824**), and F1 score (**0.1051**)*
- *The large gap between accuracy and F1 scores indicates that models mostly succeed at identifying non-entities (O tags) but struggle with actual named entities*

These challenges highlight why Twitter NER remains significantly more difficult than NER on formal text, suggesting that more sophisticated approaches beyond HMM models might be necessary for this task.

3.6 Impact of Alpha Value on F1 Score and Accuracy

The alpha parameter in HMM models controls the amount of Laplace smoothing applied to probability distributions. Based on the provided experimental results testing 13 different alpha values (from 0.1 to 1.3), we can observe several important trends:

Accuracy vs. Alpha Relationship

- *Consistent Accuracy Improvement: As alpha increases from 0.1 to 1.3, accuracy shows a steady upward trend, starting at **71.34%** and reaching **89.95%** at α=1.3.*

- *Diminishing Returns: The rate of accuracy improvement decreases at higher alpha values, with smaller gains observed between consecutive alpha values after α=0.9.*

- *Optimal Value: The highest accuracy (**89.95%**) was achieved with α=1.3, suggesting that stronger smoothing benefits overall classification performance on this dataset.*

Precision, Recall and F1 Score Trends

- *Inverse Relationship with Accuracy: While accuracy increases with alpha, precision, recall, and F1 scores generally decrease, revealing an important trade-off.*

- *Peak Performance at Lower Alpha: The highest precision (**41.24%**) was achieved at α=0.4, while the highest recall (**17.00%**) and F1 score (**14.65%**) occurred at α=0.1.*

- *Significant Drop: Between α=0.4 and α=0.5, precision drops dramatically from **41.24%** to **11.78%**, suggesting a critical threshold where the model's ability to correctly identify entities collapses.*

Analysis of the Trade-off

This pattern reveals a fundamental trade-off in NER for Twitter data:

- *Lower alpha values (less smoothing) result in models that are more aggressive in predicting named entities, leading to higher recall but lower overall accuracy due to more false positives.*

- *Higher alpha values (more smoothing) produce models that are more conservative, rarely predicting named entity tags, which improves accuracy on the predominantly*

O-tagged dataset but severely reduces the ability to detect actual entities.

The optimal alpha value depends on the specific application goals:

- *For maximizing overall accuracy: α=1.3 (**89.95%**)*
- *For maximizing F1 score: α=0.1 (**14.65%**)*
- *For balanced performance: α=0.4 (**80.39%** accuracy, **9.94%** F1)* used in final code

*This behavior aligns with the class imbalance observed in the tag distribution charts, where the vast majority of tokens (**92.5%** in test data) are non-entities (O tags). Higher alpha values bias the model toward predicting the dominant class, improving accuracy but at the expense of minority class detection.*

3.7 Results and Discussion

This section synthesizes the findings from our experiments with HMM-based Named Entity Recognition on Twitter data, examining model performance, parameter optimization, and the challenges of this domain.

Model Performance Comparison

Our experiments evaluated four HMM configurations with the following test set results:

Model	***Accuracy***	***Precision***	***Recall***	***F1 Score***
Bigram without context	0.7538	0.2062	0.0841	0.1194
Trigram without context	0.7599	0.1406	0.0824	0.1039

Bigram with context	0.7645	0.1634	0.0729	0.1009
Trigram with context	0.7782	0.1451	0.0824	0.1051

*The trigram model with contextual features achieved the highest accuracy (**77.82%**), demonstrating that higher-order models with additional contextual information can better capture the sequence patterns in Twitter data. However, the bigram model without context achieved the highest F1 score (**0.1194**), suggesting it provides a better balance between precision and recall for entity detection.*

Alpha Parameter Optimization

Our alpha parameter optimization experiments revealed a critical trade-off between accuracy and entity detection:

- *As alpha increased from 0.1 to 1.3, overall accuracy steadily improved from **71.34%** to **89.95%***
- *However, F1 scores declined dramatically, from **14.65%** at α=0.1 to **4.91%** at α=1.3*
- *Precision peaked at **41.24%** with α=0.4, then dropped sharply*

This pattern indicates that higher alpha values bias the model toward predicting the dominant non-entity (O) class, improving overall accuracy but severely reducing the ability to detect actual named entities. The optimal alpha value depends on whether the application prioritizes overall accuracy or entity detection performance.

Entity-Specific Performance

Analysis of entity-specific metrics revealed:

- *Common entities like "person", "company", and "location" achieved reasonable precision in some models (e.g., **0.71** for B-loc in bigram without context)*

- *Rare entity types like "movie", "tvshow", "sportsteam", and "musicartist" consistently showed 0.00 precision and recall across all models*

- *The "person" entity type showed the most interesting behavior with contextual information, with trigram+context models achieving higher recall (0.21) but lower precision (0.08)*

This performance variation correlates directly with the tag distribution shown in the charts, where rare entities have minimal representation in the training data.

Data Distribution Impact

The pie charts reveal extreme class imbalance:

- *Non-entity (O) tags dominate the datasets (92.5% of test data, 95.7% of training data)*

- *Among entity types, B-person, B-loc, and B-company have the highest representation*

- *Entertainment entities (movie, tvshow, musicartist) are extremely rare*

This imbalance explains why models achieve high accuracy (by predicting the dominant O class) but struggle with entity detection, resulting in low F1 scores.

Contextual Information Effects

Adding contextual features had mixed effects:

- *Improved overall accuracy for both bigram and trigram models*

- *For bigram models: Reduced precision (0.2062 → 0.1634) and recall (0.0841 → 0.0729)*

- *For trigram models: Slightly improved precision (0.1406 → 0.1451) with stable recall*

This suggests that trigram models can better leverage contextual information, while simpler bigram models may be overwhelmed by additional features.

Limitations and Future Directions

*The overall low performance metrics (best F1 score of **0.1194**) highlight the limitations of HMM approaches for Twitter NER. Future work could explore:*

1. *Advanced smoothing techniques beyond simple Laplace smoothing*

2. *Data augmentation strategies for rare entity classes*

3. *Neural approaches like BiLSTM-CRF or transformer-based models that may better handle the informal language and class imbalance*

4. *Domain adaptation techniques to leverage larger formal text NER datasets*

In conclusion, while our HMM models achieved reasonable accuracy, their ability to detect actual named entities remains limited, highlighting the significant challenges of performing NER on informal Twitter data with extreme class imbalance.